Contents

Day 9 — Data Systems: Designing the State and Information Layer

Yesterday, the focus was architecture: components, boundaries, interfaces, dependencies, state, and scaling.

Today we go deeper into one of the most consequential architectural decisions:

Where does the system put its data?

In a conventional application, this question often appears straightforward:

```
Application
     |
     v
Database
```

AI systems make the answer substantially more complicated.

A modern AI application may simultaneously need:

- relational data
- semi-structured documents
- embeddings
- caches
- large binary artifacts
- asynchronous jobs
- event streams
- conversation state
- agent execution state
- evaluation datasets
- telemetry
- model outputs

There is rarely one storage technology that is optimal for all of these.

The engineering problem is therefore not:

> "Which database should I use?"

It is:

> **"What properties does each category of data require, and which storage system provides those properties at acceptable cost and complexity?"**

This distinction is fundamental.

---

1. Data Architecture Is About Semantics

Different data systems solve different problems.

A relational database is optimized around structured records, relationships, transactions, and queries.

A vector database is optimized around approximate similarity search.

Object storage is optimized around large durable blobs.

A cache is optimized around fast, temporary access.

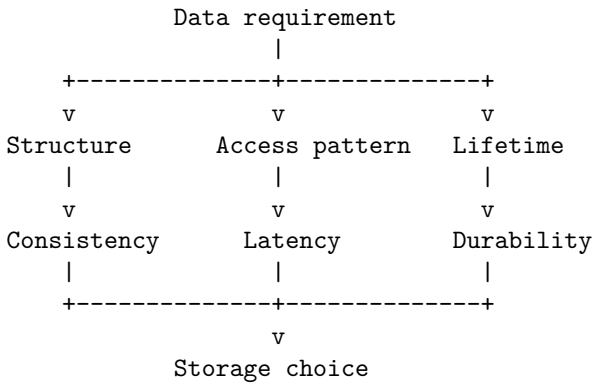
A queue is optimized around asynchronous work distribution.

An event stream is optimized around durable sequences of events consumed by multiple downstream systems.

These are not interchangeable technologies.

The architecture should begin with the **semantics of the data**, not the popularity of a particular database.

A useful abstraction is:



The database comes last.

The requirements come first.

---

2. The Data Model of an AI Application

Return to the Week 1 Personal Research Assistant.

At first glance, it appears to have "documents."

But that is not actually the application's data model.

A more complete model might be:

```
User
 |
 +-- Conversations
 |      +-- Messages
 |
 +-- Documents
 |      +-- Metadata
 |      +-- Content
 |      +-- Chunks
 |
```

```
 +-- Retrieval indexes
 |
 +-- Agent executions
 |      +-- Plans
 |      +-- Tool calls
 |      +-- Results
 |
 +-- Evaluation records
 |
 +-- Usage / billing

System
 |
 +-- Jobs
 +-- Events
 +-- Logs
 +-- Metrics
 +-- Model artifacts
```

Now the storage problem becomes clearer.

Different entities have different requirements.

For example:

Data	Primary requirement	Natural storage
Users	Transactions, relationships	Relational DB
Conversations	Durable structured state	Relational DB
Messages	Ordered durable records	Relational DB
Documents	Large immutable objects	Object storage
Document metadata	Structured queries	Relational DB
Embeddings	Similarity search	Vector index
Sessions	Very fast temporary access	Cache
Jobs	Reliable asynchronous delivery	Queue
Events	Durable event history	Event stream
Logs	High-volume append/query	Log system
Evaluation data	Structured analysis	Relational/object storage

The architecture is therefore likely to contain **multiple data systems**.

That is not automatically overengineering.

It may simply reflect the fact that the application contains different kinds of data.

---

3. Relational Databases

Relational databases remain the foundation for a large fraction of production AI applications.

Examples include PostgreSQL and MySQL.

The relational model provides:

- structured schemas
- primary keys
- foreign keys
- joins
- transactions
- constraints
- indexes
- aggregation
- strong consistency semantics

Consider:

```
users
  |
  +-- conversations
  |       |
  |       +-- messages
  |
  +-- documents
          |
          +-- document_permissions
```

This is naturally relational.

For example:

```
SELECT c.id, c.title
FROM conversations c
JOIN users u ON c.user_id = u.id
WHERE u.id = $1;
```

The relational model is particularly valuable when correctness depends on relationships.

Suppose a document belongs to a user, and that user has permission to access it.

That relationship should not merely exist as an assumption in application code.

It can be represented structurally:

```
users
  |
```

```
  | 1:N
  v
documents
  |
  | N:M
  v
permissions
```

The database can enforce some of these invariants.

---

4. Why Relational Databases Are Particularly Important for AI Systems

There is a temptation to assume that because AI applications use embeddings and unstructured text, relational databases are no longer central.

Usually the opposite is true.

The AI components generate additional metadata that needs conventional persistence.

For example:

```
model invocation
    |
    +-- user_id
    +-- conversation_id
    +-- model
    +-- prompt_tokens
    +-- completion_tokens
    +-- latency
    +-- cost
    +-- trace_id
    +-- evaluation_result
```

These are structured records.

They need:

- filtering
- aggregation
- reporting
- relationships
- transactions
- access control

A relational database is often an excellent fit.

5. Document Databases

Document databases store records as flexible documents, commonly represented as JSON-like structures.

They are useful when:

- schemas evolve rapidly
- records are naturally hierarchical
- fields vary substantially between records
- joins are limited or undesirable
- access patterns are primarily document-oriented

For example, an agent execution might look like:

```
{
  "execution_id": "exec_123",
  "status": "completed",
  "plan": [
    {
      "step": 1,
      "tool": "search",
      "status": "completed"
    },
    {
      "step": 2,
      "tool": "summarize",
      "status": "completed"
    }
  ],
  "metadata": {
    "model": "..."
  }
}
```

This can be a natural document.

But flexible schemas do not mean "schema doesn't matter."

A document database can still have:

- implicit schemas
- versioning requirements
- migration problems
- indexing requirements
- consistency requirements

The absence of a formal relational schema does not eliminate data modeling.

It simply moves more of the responsibility into the application.

---

6. Relational vs Document

The decision should be based on access patterns.

Suppose you have:

```
Conversation
    +-- metadata
    +-- messages
    +-- participants
    +-- permissions
```

If you frequently need:

```
Find all conversations
belonging to users
who have access to document X
created after date Y
with messages matching condition Z
```

relational modeling is attractive.

If instead your dominant operation is:

```
Load complete execution record by execution_id
```

a document model may be appropriate.

The question is not:

> "Are my data structures JSON?"

The question is:

> **"What queries and invariants define the system?"**

---

7. Vector Databases

AI applications introduce a new access pattern:

> Find objects that are semantically similar to this query.

Traditional relational indexes answer questions like:

```
WHERE user_id = 42
WHERE timestamp > ...
WHERE status = 'active'
```

Vector search answers something fundamentally different:

```
Find the k vectors closest to this query vector.
```

Represent a document chunk as:

```
text
  ↓
embedding model
  ↓
vector in R^d
```

For example:

```
[-0.12, 0.41, 0.07, ...]
```

Then retrieval becomes a nearest-neighbor problem.

Common similarity measures include:

- cosine similarity
- dot product
- Euclidean distance

A vector database or vector index provides infrastructure for performing these searches efficiently.

---

8. Vector Search Is Not a Database Replacement

One of the most important architectural lessons is:

> **A vector index is not a replacement for your system of record.**

Consider a document chunk:

```
chunk_id
document_id
text
embedding
page
timestamp
permissions
```

The vector index may primarily care about:

```
chunk_id
embedding
```

But the application still needs:

```
document ownership
access permissions
document metadata
version
source location
```

A common architecture is therefore:

```
                Query
                  |
                  v
            Vector Search
                  |
                  v
             chunk_ids
                  |
                  v
          Relational DB
                  |
                  v
      Metadata / permissions
                  |
                  v
            Final context
```

The vector index becomes a **derived data structure**.

This distinction is extremely important.

If the vector index can be reconstructed from authoritative data, it becomes much easier to reason about correctness and recovery.

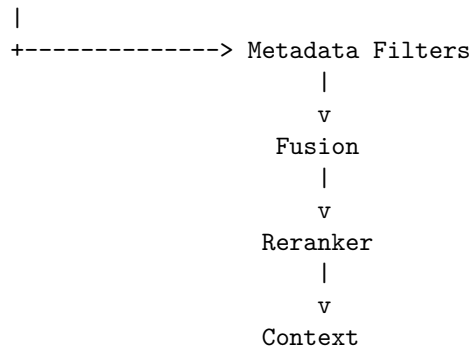
---

9. Hybrid Retrieval

Real retrieval systems frequently combine multiple retrieval mechanisms.

For example:

```
Query
 |
 +--------------> Keyword Search
 |
 +--------------> Vector Search
```



This illustrates an important principle:

> **Storage architecture should follow retrieval architecture.**

If your application needs:

- lexical search
- semantic search
- metadata filtering
- temporal filtering
- permissions filtering

then one index may not be sufficient.

---

10. Caches

A cache is fundamentally different from a database.

A database answers:

> "What is the authoritative state?"

A cache answers:

> "Can I retrieve this value cheaply because I expect it to be useful again?"

Typical cache data includes:

```
session state
retrieval results
model responses
embeddings
configuration
rate-limit counters
computed features
```

The key architectural property is that cached data should generally be **reconstructible**.

If deleting the cache destroys the system, you did not build a cache.

You built a database with questionable durability.

---

11. Cache Invalidation

Caching introduces one of the oldest problems in computer science:

> **When is cached data no longer valid?**

Suppose:

```
Document
   |
   v
Embedding
   |
   v
Vector index
   |
   v
Cached retrieval result
```

The document changes.

Now three layers may contain stale data.

This creates a dependency graph:

```
Document
   ↓
Embedding
   ↓
Vector Index
   ↓
Retrieval Cache
   ↓
Generated Answer Cache
```

A change at the beginning may invalidate everything downstream.

AI systems can therefore have surprisingly complicated cache invalidation problems.

---

12. Object Storage

Object storage is optimized for large immutable or semi-immutable objects.

Examples include:

- PDFs
- images
- audio
- video
- datasets
- model artifacts
- generated reports
- evaluation traces

Instead of storing a 50 MB PDF directly inside a relational database, you might store:

```
Object Storage
    |
    +-- documents/user123/paper.pdf
```

and put metadata in the relational database:

```
documents

id
user_id
object_key
filename
mime_type
size
created_at
checksum
```

This creates a clean separation:

```
Relational DB
    = metadata and relationships

Object Storage
    = large content
```

---

13. Why Object Storage Matters for AI

AI systems generate large artifacts.

Consider an agent that produces:

- a research report
- intermediate files
- screenshots
- datasets
- extracted tables
- audio
- model outputs

These should not necessarily flow through the transactional database.

Object storage provides:

- high durability
- large capacity
- relatively low cost
- lifecycle management
- versioning
- access control

This makes it an important architectural primitive for AI systems.

---

14. Queues

A queue represents work that needs to be performed.

For example:

```
Document uploaded
       |
       v
      Queue
       |
       v
Ingestion worker
```

The queue decouples the producer from the consumer.

The producer does not need to know:

- which worker handles the job
- when it runs
- how many workers exist
- whether processing takes 1 second or 10 minutes

This is a powerful architectural boundary.

---

15. Queues Are About Work, Not Facts

This distinction is subtle but important.

A queue usually represents:

> **"Something needs to be done."**

An event stream represents:

> **"Something happened."**

For example:

```
Queue:
    process_document(document_id)

Event:
    document_processed(document_id, version=4)
```

These may look similar but have different semantics.

A queue is typically consumed to make progress.

An event may be retained so multiple systems can react independently.

---

16. Event Streams

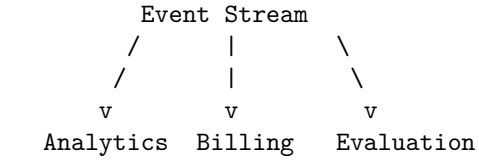
An event stream is a durable sequence of events.

For example:

```
Event Stream

t1  UserCreated
t2  DocumentUploaded
t3  DocumentIndexed
t4  QueryExecuted
t5  AnswerGenerated
t6  EvaluationCompleted
```

Different consumers can independently process the stream:



This provides an important form of decoupling.

The producer does not need to know all current or future consumers.

This becomes valuable as systems grow.

---

17. Event-Driven AI Systems

Imagine the research assistant emits:

```
DocumentUploaded
DocumentParsed
DocumentEmbedded
DocumentIndexed
QueryReceived
RetrievalCompleted
AgentStarted
ToolCalled
AnswerGenerated
EvaluationCompleted
```

Now many systems can consume these events.

For example:

```
Evaluation service
      ^
      |
Query events -----> Analytics
      |
      +------------> Cost accounting
```

The agent runtime does not need direct knowledge of all of these systems.

This is one of the architectural benefits of events:

> **New consumers can be added without modifying the producer.**

---

18. Choosing the Right Data System

A useful decision framework is to ask seven questions.

1. What is the data?

Structured records?

Large blobs?

Embeddings?

Temporary state?

Events?

Work items?

2. What is the access pattern?

```
lookup
range query
join
aggregation
similarity search
append
stream
queue
```

3. What consistency is required?

Does every reader need the newest value immediately?

Or is eventual consistency acceptable?

4. What latency is required?

```
milliseconds
seconds
minutes
hours
```

5. How durable must the data be?

Can it be reconstructed?

Can it be lost?

Must it survive regional failure?

6. How much data will exist?

Megabytes?

Terabytes?

Petabytes?

7. What happens when the system fails?

Can the data be rebuilt?

Can processing resume?

Can messages be replayed?

These questions usually lead toward an appropriate technology.

---

19. A Practical Decision Matrix

Requirement	Relational DB	Document DB	Vector DB	Cache	Object Storage	Queue	Event Stream
Transactions	***	**	*	—	—	—	—
Relationships	***	*	*	—	—	—	—
Flexible schema	**	***	**	***	***	**	**
Similarity search	*	*	***	*	—	—	—
Very low latency	**	**	**	***	*	—	**
Large blobs	*	*	—	—	***	—	—
Async work	—	—	—	—	—	***	**
Durable history	***	***	**	*	***	*	***
Replay	*	*	—	—	**	*	***

The stars are not universal performance claims.

They are a reasoning aid.

The important exercise is to understand **why** a technology receives a high or low rating for a particular requirement.

---

20. The AI Architecture Challenge

Now comes the most important exercise of Day 9.

Do not design the architecture yourself first.

Instead:

> **Have the AI agent design it.**

Give the agent the Week 1 application specification.

Ask it to produce:

1. data model
2. storage architecture
3. database choices
4. caching strategy
5. queue architecture
6. event architecture
7. consistency model
8. indexing strategy
9. backup and recovery strategy
10. scaling strategy

Give it realistic constraints.

For example:

```
Application:
Personal Research Assistant

Users:
1 million

Documents:
100 million

Average document:
2 MB

Queries:
100 requests/sec average
1,000 requests/sec peak

Requirements:
- multi-tenant
- document permissions
- conversational state
- semantic retrieval
- keyword retrieval
- asynchronous ingestion
- auditability
- 99.9% availability
```

Then ask the agent:

> **"Design the complete data architecture and explain why each technology was selected."**

Do not immediately accept the answer.

That is not the exercise.

The exercise begins when the agent finishes.

---

21. Critique the Agent's Architecture

Your role is now to attack the design.

Ask:

Assumption 1

Why does it need a vector database?

Could PostgreSQL with a vector extension be sufficient?

Assumption 2

Why does it need a document database?

Could the relational schema handle the workload more simply?

Assumption 3

Why is Redis required?

What data is actually being cached?

What invalidates it?

Assumption 4

Why is an event stream required?

Would a queue be sufficient?

Assumption 5

What is the system of record?

If the vector index disappears, can the system reconstruct it?

Assumption 6

What happens if the queue delivers the same job twice?

Assumption 7

What happens if indexing succeeds but the database transaction fails?

Assumption 8

What happens if the database succeeds but the event is never published?

Assumption 9

What happens when the schema changes?

Assumption 10

What happens when the system grows by 100×?

These questions expose architectural assumptions.

---

22. AI-Generated Architecture Is Not Automatically Good Architecture

This exercise introduces a critical engineering skill.

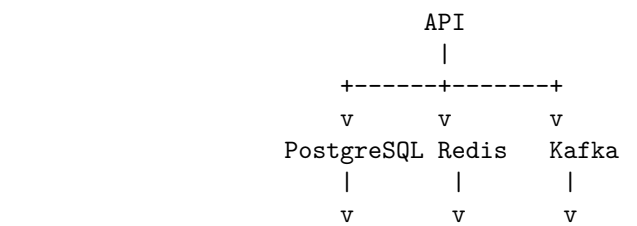
LLMs are extremely good at producing plausible architecture diagrams.

They know that production systems often contain:

```
PostgreSQL
Redis
Kafka
S3
Vector DB
Kubernetes
API Gateway
Load Balancer
```

The danger is that they can assemble these technologies into an architecture that **looks professional without being justified**.

For example:



```
VectorDB  Search   Workers
    |
    v
   S3
```

It looks sophisticated.

But the architecture might be completely unnecessary.

Perhaps PostgreSQL could handle:

- relational data
- metadata
- vector search
- transactional state

and a simple queue plus object storage could handle the rest.

The agent may have optimized for **architectural familiarity rather than system requirements**.

This is exactly the failure mode you are learning to detect.

---

23. Architecture Review as Adversarial Reasoning

A strong architecture review should therefore proceed like a security review.

Do not ask:

> "Does this look reasonable?"

Ask:

> **"What assumption would have to be false for this architecture to fail?"**

For every major component, identify:

```
Component
   ↓
Purpose
   ↓
Assumption
   ↓
Failure mode
   ↓
Consequence
```

```
   ↓
Mitigation
```

For example:

```
Vector DB
   ↓
Fast semantic retrieval
   ↓
Assumes index remains available
   ↓
Index outage
   ↓
Retrieval unavailable
   ↓
Fallback / rebuild strategy
```

This turns architecture review into an engineering discipline rather than an aesthetic exercise.

---

24. The Data Lifecycle

One of the most useful ways to reason about data architecture is to follow a piece of data through its lifecycle.

Consider a PDF.

```
Upload
  |
  v
Object Storage
  |
  v
Metadata DB
  |
  v
Queue
  |
  v
Parser
  |
  v
Chunker
  |
  v
Embedding Model
```

```
  |
  v
Vector Index
  |
  v
Retrieval
  |
  v
LLM Context
  |
  v
Answer
  |
  v
Evaluation
```

At every stage ask:

- Is this data authoritative?
- Is it derived?
- Is it durable?
- Can it be recomputed?
- Who owns it?
- How is it versioned?
- How is it invalidated?
- What happens if this stage fails?

This produces a much more precise architecture than simply listing technologies.

---

25. The System of Record Principle

Every important piece of information should have a clear authoritative source.

For example:

```
Document content
    → Object Storage

Document metadata
    → Relational DB

Conversation state
    → Relational DB

Embedding
    → Vector Index
```

```
Cache
    → Cache system
```

The distinction between **source of truth** and **derived state** dramatically simplifies recovery.

Suppose the vector database is destroyed.

If embeddings are derived from:

```
Object Storage
       +
Metadata DB
```

then rebuilding the vector index becomes a batch processing problem.

Without a source of truth, the loss of the vector database may mean permanent data loss.

The principle is:

> **Derived state should be disposable whenever practical.**

---

26. Data Architecture and Reliability

Yesterday we discussed reliability at the architectural level.

Today we can make it concrete.

Suppose the architecture is:

```
Document
   ↓
DB
   ↓
Queue
   ↓
Embedding Worker
   ↓
Vector DB
```

Now consider failures.

Database succeeds, queue fails

The document exists, but ingestion never starts.

Queue succeeds, worker crashes

The job must be retried.

Worker succeeds, acknowledgment fails

The job may execute twice.

Vector DB fails

The document is stored but unavailable for semantic retrieval.

Embedding model changes

Existing embeddings may become incompatible with the new model.

Each failure implies a data-system requirement.

This is why data architecture and reliability architecture cannot be separated.

---

27. Data Versioning

AI systems make versioning unusually important.

Consider embeddings.

An embedding depends on:

```
document
embedding model
model version
preprocessing
chunking strategy
```

Therefore:

```
embedding_id
document_id
embedding_model
embedding_version
chunking_version
created_at
```

may matter.

Suppose you switch from:

```
EmbeddingModel V1
```

to:

```
EmbeddingModel V2
```

You now have a migration problem.

Do you:

```
V1 → V2
```

all documents immediately?

Or:

```
V1
 |
 +-- continue serving
 |
 +-- gradually generate V2
```

This is not merely an ML problem.

It is a **data architecture problem**.

---

28. The Architecture You Should End the Day With

Your final design should look less like:

```
"Let's use PostgreSQL + Redis + Kafka + Pinecone + S3."
```

and more like:

```
Requirement
    ↓
Data semantics
    ↓
Access pattern
    ↓
Consistency requirement
    ↓
Durability requirement
    ↓
Scale requirement
    ↓
Failure model
    ↓
Technology choice
```

For example:

```
Large immutable document
        ↓
Object storage

Transactional metadata
        ↓
Relational DB

Semantic retrieval
        ↓
Vector index

Temporary hot result
        ↓
Cache

Long-running processing
        ↓
Queue

Durable sequence of system events
        ↓
Event stream
```

This is the core discipline.

---

29. Key Takeaways

1. **Data architecture begins with data semantics, not technology selection.**
2. **Different storage systems optimize for different access patterns.** Relational databases, vector indexes, caches, object storage, queues, and event streams solve different problems.
3. **Relational databases remain foundational to AI applications.** Users, permissions, conversations, metadata, billing, evaluations, and operational state are often highly relational.
4. **Vector databases solve similarity-search problems; they do not replace the system of record.**
5. **Object storage is the natural home for large durable artifacts.** Keep large content separate from transactional metadata.
6. **Caches contain derived, disposable state.** If losing the cache destroys

the system, it is not really functioning as a cache.

7. **Queues represent work; event streams represent facts about what happened.** Understanding this distinction prevents many architectural mistakes.
8. **Every important piece of data should have a clear source of truth.**
9. **Derived state should be reconstructible whenever practical.**
10. **AI systems require explicit consideration of versioning.** Models, embeddings, chunking strategies, prompts, and retrieval indexes can all introduce data-version dependencies.
11. **Data architecture is reliability architecture.** Retries, duplicate processing, partial failure, consistency, recovery, and replay all depend on how data moves through the system.
12. **Do not accept an AI-generated architecture because it looks sophisticated.** LLMs are very good at producing plausible architectures and very capable of introducing unjustified complexity.
13. **The engineer's job is increasingly to critique AI-generated designs.** Ask what assumptions the architecture makes, what happens when those assumptions fail, and whether each component is actually necessary.
14. **The recurring engineering pattern is:**

```
AI proposes
    ↓
Engineer interrogates
    ↓
Engineer identifies assumptions
    ↓
Engineer tests assumptions
    ↓
Engineer redesigns
```

This is becoming one of the defining skills of AI-era engineering.

The important lesson of Day 9 is therefore:

> **Do not ask an AI which database to use. Ask it to design a data architecture—and then make it defend every architectural decision.**

The AI can generate the first draft.

You are responsible for the architecture.